

Heat Distribution Optimization: Control Theory on the Heat Equation

Antoine Levrat

February 2026

Résumé

This project presents the numerical solution of a thermal control problem. We put ourselves in a steady state regime, the objective is to determine a heating distribution u that minimizes the deviation from a target temperature y_d first, under the constraint of a Poisson-type equation, and then on more complex PDE's. The resolution is based on the adjoint state method and a projected gradient algorithm with an Armijo line search.

1 Problem Formulation

1.1 Objective and Control Variable

We seek to minimize the cost functional $J(u)$ defined by :

$$J(u) = \frac{1}{2} \int_{\Omega} (y - y_d)^2 dx + \frac{\alpha}{2} \int_{\Omega} u^2 dx \quad (1)$$

Where $y(u)$ is the state of the system depending on the control u , y_d is the target temperature, and $\alpha > 0$ is the regularization parameter. The control u belongs to the set of admissible controls $\mathcal{U}_{ad}$, defined by local bounds :

$$\mathcal{U}_{ad} = \{u \in L^2(\Omega) \mid u_{min} \leq u(x) \leq u_{max} \text{ a.e. in } \Omega\} \quad (2)$$

In this paper we will work with $\Omega \in \mathbb{R}^2$.

Remark : this work is done with J being a "least square" function, where you want to reach a target. In PDE constrained optimization, another class of problem exists : "compliance" problems. In many cases, these have the benefit of being self-adjoint, thus needing simpler algorithms. While this will not be detailed here, our case can't apply these principles, hence the reason for another method.

1.2 State Equation

The state y is governed by the Poisson equation with homogeneous Dirichlet boundary conditions :

$$\begin{cases} -\Delta y = u & \text{in } \Omega \\ y = 0 & \text{on } \partial\Omega \end{cases} \quad (3)$$

Matplotlib 3D representation of one heat repartition obtained with the Algorithm below in a control set : $\mathcal{U}_{ad} = \{u \in L^2(\Omega) \mid 0 \leq u(x) \leq 6 \text{ a.e. in } [0, 1]^2\}$

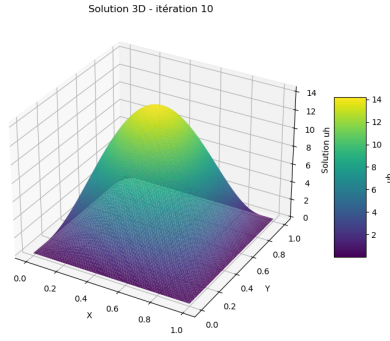


Figure 1 – Without control set

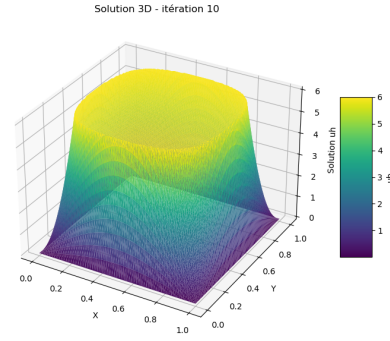


Figure 2 – With control set

2 Existence and Uniqueness Proof

We will not go into many details as this is not the goal of this project but in a few words :

- **Existence** : The functional $J(u)$ consists of a least-squares term and an L^2 norm. Since the mapping $u \mapsto y$ is affine and continuous, J is a convex, continuous, and coercive functional on $\mathcal{U}_{ad}$.
- **Uniqueness** : Due to the regularization term $\frac{\alpha}{2}\|u\|^2$, J is strictly (and even strongly) convex. As $\mathcal{U}_{ad}$ is a non-empty closed convex set of $L^2(\Omega)$, there exists a unique minimizer u^* .

3 Analysis and Algorithm

3.1 Expression of $J'(u)$

We first introduce the lagrangian of this problem :

$$\mathcal{L} : U_{ad} \times H_0^1(\Omega) \times H_0^1(\Omega) \rightarrow \mathbb{R}$$

such that :

$$\mathcal{L}(u, y, p) = \frac{1}{2} \int_{\Omega} (y - y_d)^2 dx + \frac{\alpha}{2} \int_{\Omega} u^2 dx + \int_{\Omega} \nabla y \nabla p - u \cdot p dx$$

The gradient of the functional J is obtained as follows :

$$\partial_u \mathcal{L} = J'(u) = \alpha u - p$$

and p is the adjoint state, solution to :

$$\partial_y \mathcal{L} = 0 \Leftrightarrow \begin{cases} -\Delta p = -(y - y_d) & \text{in } \Omega \\ p = 0 & \text{on } \partial\Omega \end{cases}$$

Remark : To compute the gradient of J from a given u , one must first solve the state equation to get y , and then solve the adjoint equation to get p .

3.2 Numerical Algorithm

3.2.1 Projected Gradient

To handle the bound constraints on u , we use the projection operator $P_{\mathcal{U}_{ad}}$ defined by point-wise truncation :

$$P_{\mathcal{U}_{ad}}(v)(x) = \max(u_{min}, \min(u_{max}, v(x))) \quad (4)$$

The algorithm begins with an initial selected u_0 .

The iterations are given by $u_{n+1} = P_{\mathcal{U}_{ad}}(u_n - \mu_n J'(u_n))$. Where $J'(u_n)$ is calculated solving the state and the adjoint equation as discussed previously, and μ_n is the step size. Solving an EDP has a great cost, hence we will focus our attention on ways to decrease the number of iterations.

3.2.2 Armijo and Backtracking

To ensure the decrease of J an Armijo line search is used. The step size μ_n is determined such that :

$$J(u_{n+1}) \leq J(u_n) + \sigma \mu_n \langle J'(u_n), u_{n+1} - u_n \rangle \quad (5)$$

If the condition is not met, the step size is reduced : $\mu \leftarrow \gamma \mu$ with $\gamma \in]0, 1[$. In the implementation of this method, we've chosen to keep the last successful step for the next u_{n+1} and increase the step by some factor if too many Armijo conditions were met on the first try successfully.

3.2.3 Barzilai-Borwein method

In a second time, we've tried to accelerate the optimizer. One simple and effective method has been the *Barzilai-Borwein one's*. The Goal is to approximate Newton's step. This method is a derivation of the secant method, we here approximate the hessian matrix as :

$$(u_n - u_{n-1})H_n \approx \nabla J(u_n) - \nabla J(u_{n-1})$$

this gives the relation :

$$\alpha = \frac{1}{H_n} = \frac{\langle s_k, s_k \rangle}{\langle s_k, y_k \rangle}$$

Where $s_k = u_n - u_{n-1}$ and $y_k = \nabla J(u_n) - \nabla J(u_{n-1})$.

Remark : This method will prove to be very effective, but it is less stable in some cases. We have chosen to verify Armijo to help when dealing with non regular functions, resulting almost always in a condition met on the first check. But this limits the steps to always decrease the function J . Thus the implementation of a non-linear Armijo condition would be more adequate in this scope.

4 Numerical results and convergence

In this section we test the algorithm on $\Omega = [0, 1]^2$, with $y_d = \sin(\pi x) \sin(\pi y)$. When $\mathcal{U}_{ad}$ is sufficiently big, this regular solution allows us to determine the expected solution $u_{exact} = \frac{2\pi^2}{1+\alpha(2\pi^2)^2} \sin(\pi x) \sin(\pi y)$

All tests below in this section have been done with : $\alpha = 10^{-3}$, $|u_{min} = -u_{max} = -20$, $|\gamma_+ = 1.2, \gamma_- = 0.5$, $|\mu_0 = 5.0$

4.1 Comparison of the 2 models

We define here 2 methods :

- Method 1 : Projected gradient using Armijo and backtracking (std) as a source of step adaptability
- Method 2 : Projected gradient using Armijo but with Barzilai-Borwein (BB) step

Iterative convergence analysis for Method 1 and 2

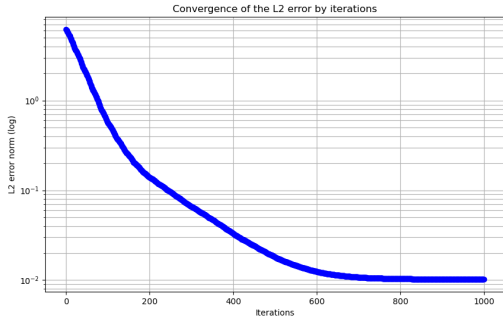


Figure 3 – Iterative convergence for Method 1

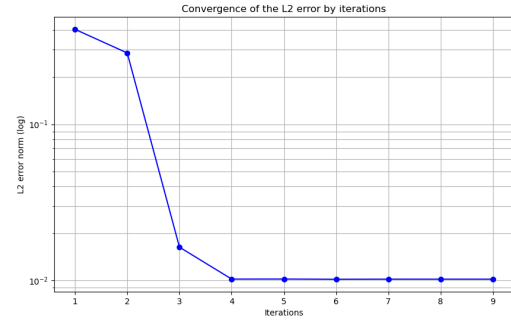


Figure 4 – Iterative convergence for Method 2

Time and mesh refinement analysis

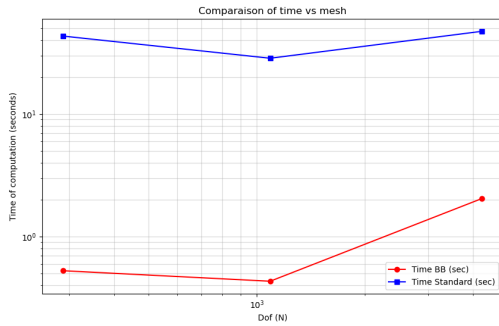


Figure 5 – Log graph of the time for computation per iterations for Methods 1 (std) and 2 (BB)

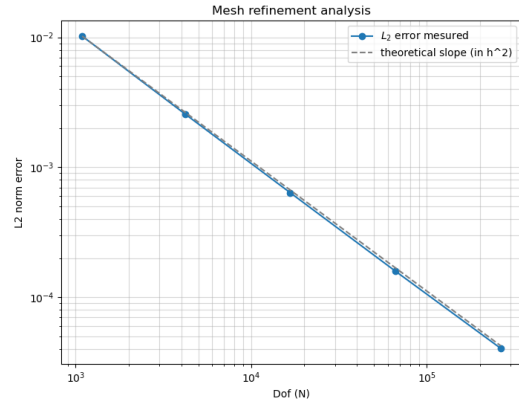


Figure 6 – log graph of the L2 error compared to mesh refinement for Method 2

The performance and convergence properties of the two projected gradient methods were evaluated throughout numerous tests. As shown in Figure 3 and Figure 4, both methods successfully converge to an L^2 error norm below 10^{-2} . However, the Barzilai-Borwein (BB) approach demonstrates a significantly higher efficiency. While the standard Armijo backtracking (Method 1) requires approximately 800 iterations to reach the error floor, the BB-enhanced gradient (Method 2) achieves the same level of precision in fewer

than 5 iterations. (note : Armijo parameters can be adjusted to converge a bit faster but not significantly)

Computational Time :

Figure 5 highlights the stark difference in computational cost. Across all tested mesh resolutions ($[16^2, 32^2, 64^2]$), Method 2 (BB) is orders of magnitude faster than Method 1. For instance, at a resolution of 32, the BB method completes in ≈ 0.43 seconds, whereas the standard method takes ≈ 28.55 seconds. This radical disparity stems from the BB method's ability to utilize a more aggressive, quasi-Newton-like step size, drastically reducing the total number of iterations and backtracking steps.

Mesh Refinement and Accuracy :

Finally, Figure 6 confirms the numerical consistency of the implementation. The L^2 error follows the theoretical h^2 convergence slope as the mesh is refined from 32 to 512. The measured errors, ranging from 1.02×10^{-2} to 4.04×10^{-5} , perfectly align with the expected decay rate, validating the reliability of the solver regardless of the step-size strategy used.

Implementation note : The code used here was made in python using fenics library. When testing, in some great order iteration, using the projected gradient, the code would hard crash.

This problem was caused because when $u_{k+1} \leftarrow u - \mu J'(u_k)$, applying the projection : $\max(\min(u_{k+1}))$ if one doesn't store the current value of u_{k+1} , then fenics tries to do $\max(\min(\max(\min(\dots(u_0))))))$. Therefore one need to interpolate the solution.

This wasn't the only problem seen in this matter, but since it took a great amount of dedication to find the issue, it felt right to discuss it here.

4.2 Scaling analysis

One major utility of using fenics, is the use of MPI, allowing computation on multiple cores. Although the code wasn't made to be efficient in this regard, it is still interesting to test it.

The parallel efficiency of the implementation was evaluated through strong and weak scaling tests, as shown in Figure 7. The experiments were conducted on a personal computer not specifically designed for this application, this will prove to cause some problems.

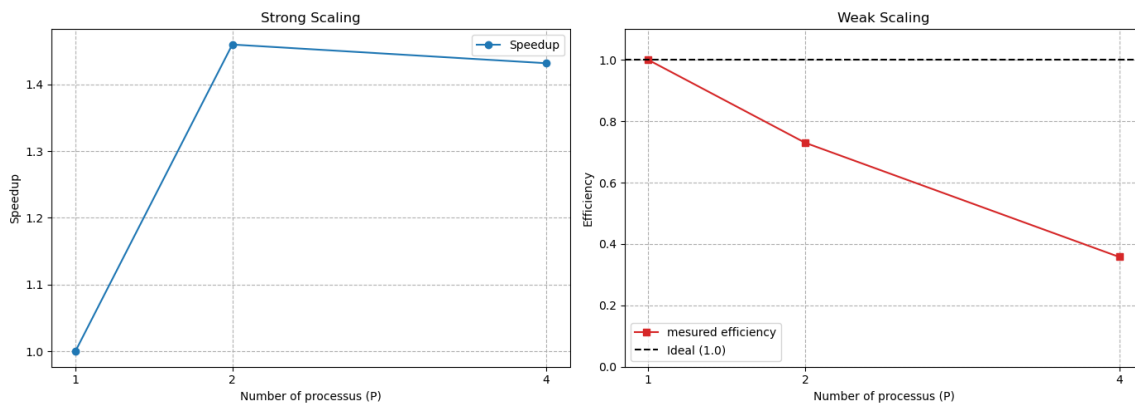


Figure 7 – Strong and Weak scaling analysis

- **Strong Scaling :** The speedup increases when moving from 1 to 2 processes, reaching a peak of approximately 1.45. However, beyond 2 processes, the speedup

plateaus and slightly declines.

Remark : in those tests the mesh was $nx = ny = 512$

- **Weak Scaling** : The efficiency shows a significant drop as the number of processes increases, falling to roughly 0.36 for 4 processes.

Remark : in those test the mesh started at 512^2 and got twice as big as the number of processes doubled

These result can be partially explained by the RAM being saturated for the 4 processes tests. One could try these on a cluster made for this purpose to rectify to an extend those graphs.

4.3 Visualization

Some visuals are needed to show the iterations. The visuals have been done on *paraview* with Method 1 on iterations from 0 to 20 on a 128^2 mesh and $\mathcal{U}_{\text{ref}}$ not restricting the solution u .

Iterative convergence visuals method 1

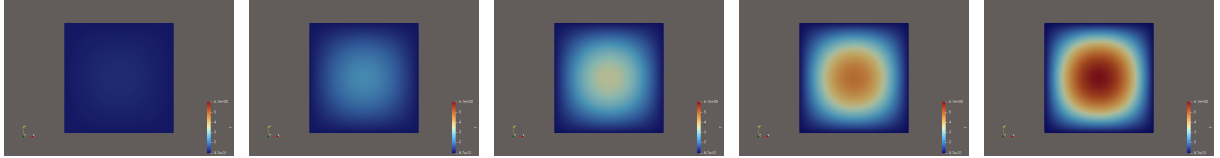


Figure 8 – **Figure 9** – **Figure 10** – **Figure 11** – **Figure 12** –
Iter=0 Iter=5 Iter=10 Iter=15 Iter=20

5 Extension of the PDE Model

5.1 PDE with an advection term

To extend the work, and discover more of the fenics library, we have decided to add an advection term to the state equation. This requires us to find the new adjoint state equation.

Define the state as :

$$\begin{cases} -\Delta y + v \cdot \nabla y = u & \text{in } \Omega \\ y = 0 & \text{on } \partial\Omega \end{cases} \quad (6)$$

Through a similar reasoning as in part 3.1 we can deduce :

$$\begin{cases} -\Delta p - v \cdot \nabla p = -(y - y_d) & \text{in } \Omega \\ p = 0 & \text{on } \partial\Omega \end{cases} \quad (7)$$

In this case $J'(u)$ doesn't change from it's past form.

Here again we include some visuals, those have been done with $v = (10, 0)$, on the projected gradient algorithm from iteration 0 to 30 :

Iterative convergence visuals method 1

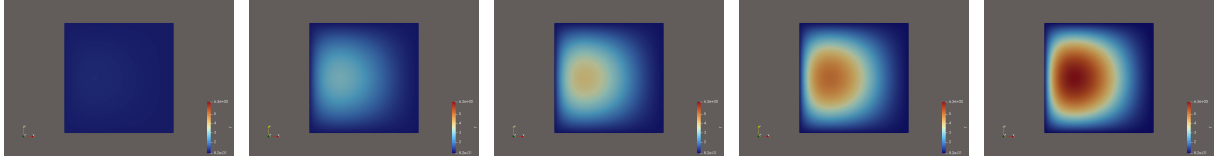


Figure 13 – **Figure 14** – **Figure 15** – **Figure 16** – **Figure 17** –
Iter=0 Iter=10 Iter=15 Iter=20 Iter=30

5.2 Time dependant problems

After this successful implementation, we have tried to add a derivative in time, in order to get closer and closer to a complete model.

Define the new state as :

$$\begin{cases} \partial_t y - \Delta y + v \cdot \nabla y = u & \text{in } \Omega \times [0, T] \\ y = 0 & \text{on } \partial\Omega \times [0, T] \\ y(x, 0) = y_0(x) = 0 \text{ in this tests} & \text{in } \Omega \end{cases} \quad (8)$$

The adjoint is then defined as

$$\begin{cases} -\partial_t p - \Delta p - v \cdot \nabla p = -(y - y_d) & \text{in } \Omega \times [0, T] \\ p = 0 & \text{on } \partial\Omega \times [0, T] \\ p(x, T) = 0 & \text{in } \Omega \end{cases} \quad (9)$$

In this case, the function we now minimize over $\mathcal{U}_{\text{aff}}$ becomes :

$$J(u) = \frac{1}{2} \int_0^T \int_{\Omega} (y - y_d)^2 dx + \frac{\alpha}{2} \int_0^T \int_{\Omega} u^2 dx \quad (10)$$

A first remark could be made, the state need to be solved from $t = 0$ whereas the adjoint is in reverse order.

For the time dependency, we have chosen to keep it as simple as possible and implement a backward Euler method (*which has the advantage of being stable in all conditions i.e no CFL condition*). For example the state writes :

$$\partial_t y \approx \frac{y_{n+1} - y_n}{\delta t} = \Delta y_{n+1} - v \cdot \nabla y_{n+1} + u_{n+1}$$

This adds complexity to the solver and increases the cost of the optimization process. Since we now need to solve 2 PDE's (state and adjoint) for each time interval. For the computation of J we have decided to do the integration in time using trapezoid method.

After redefining most of our python functions, the results are displayed in this section :

Iterative convergence visuals method 1

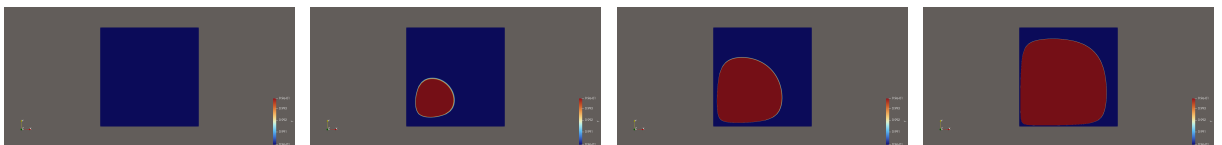


Figure 18 – $t=0$ **Figure 19** – $t=dt$ **Figure 20** – $t=2*dt$ **Figure 21** – $t=6*dt$

Iterative convergence visuals method 2

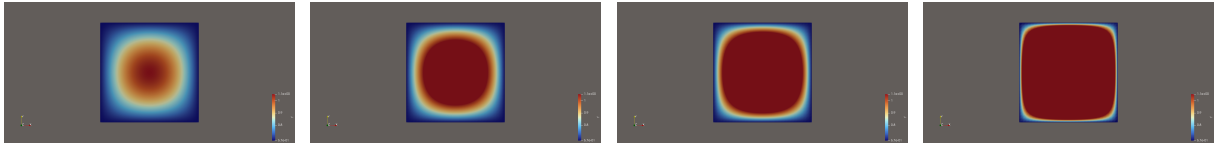


Figure 22 – $t=0$ **Figure 23** – $t=dt$ **Figure 24** – $t=2*dt$ **Figure 25** – $t=6*dt$

About these results : In these tests, u_{init} was the constant function equal to 1 everywhere. $y_d = t \sin(\pi x) \sin(\pi y)$. Mesh was $[0, 1]^2$ on a 32×32 grid. Concerning time, we evaluated the functions from $t = 0$ to $t = 1$ with 20 steps. For Method 1 results, advection was tested with success with $v = [10, 10]$.

This of course is still an optimization problem, displayed here for method 1 is after 19 line search iterations, and for Method 2, 3 line search iterations.

Remark : Some tests allowed us to guarantee to an extent these results, in this setting, Method 1, after 400 iterations, the 5 first digits of the function J stayed the same. Method 2 only took 5 iterations to reach the same results as Method 1. (The 2 values were the same). But because of the complexity of the system and a lack of time, we were not able to confirm with the same precision as in part 4 the exactness of the results.

6 Conclusion

This project successfully achieved the numerical solution of an optimal thermal control problem governed by a Poisson equation. The model was further extended to include advection-diffusion terms and time-dependent systems. The implementation of the adjoint state method, combined with the Barzilai-Borwein algorithm, demonstrated superior efficiency. It drastically reduced the number of iterations—from approximately 800 in the standard projected gradient method to fewer than 10—to reach the same error floor.

The temporal extension, using a backward Euler scheme, confirmed the robustness of the approach despite the increased computational cost required to solve the state and adjoint equations at each time step. The visual results validate to an extent the solver's ability to reach a target temperature y_d in a stable and precise manner. Furthermore, in simpler settings, the L^2 error convergence slope, confirm the reliability of the solver regardless of the step-size strategy. This said, more test should be done to ensure these result were accurate in all settings.

This work provides a solid foundation for more complex multi-physics simulations using the FEniCS library

The Project can be found at :

https://github.com/antoinelevrat78-cyber/Opti_EDP